

CO1-AT-1

MLA0201-Fundamentals Of Machine Learning

Q1. Implement the Concept of Probability Distribution to Predict the Likelihood of Spam Emails in a Mail Filtering System**Definition:**

A Probability Distribution is a mathematical function that describes the likelihood of different possible outcomes. In spam email filtering, probability distribution is used to determine whether an email belongs to the Spam or Not Spam (Ham) category based on the occurrence of words and features in the email.

The Naïve Bayes Classifier is widely used because it assumes that all features (words) are independent and calculates the probability of an email being spam.

Mathematical Formula:

Using Bayes' Theorem:

$$P(\text{Spam} | \text{Email}) = \frac{P(\text{Email} | \text{Spam}) \times P(\text{Spam})}{P(\text{Email})}$$

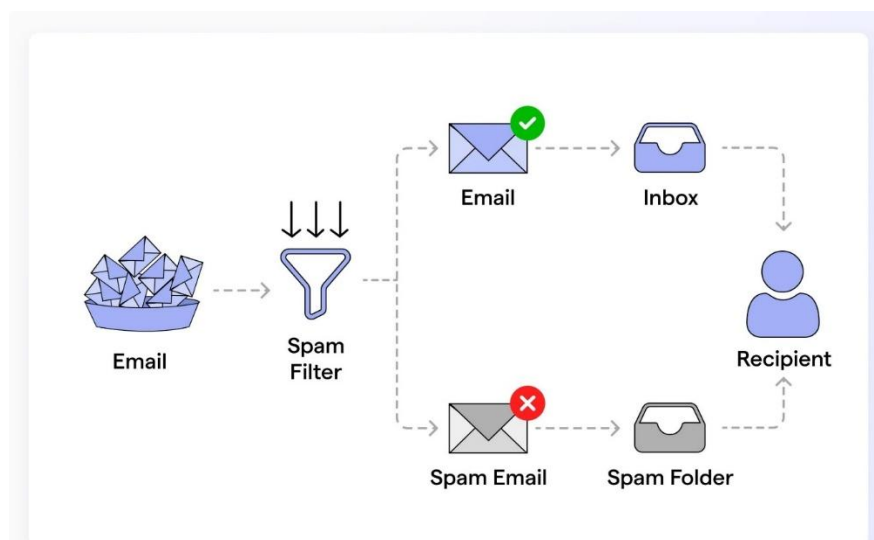
Where:

- **P(Spam | Email)** = Probability that the email is spam
- **P(Email | Spam)** = Probability of the email features appearing in spam emails
- **P(Spam)** = Prior probability of spam emails
- **P(Email)** = Overall probability of the email

If:

$$P(\text{Spam} | \text{Email}) > P(\text{Ham} | \text{Email})$$

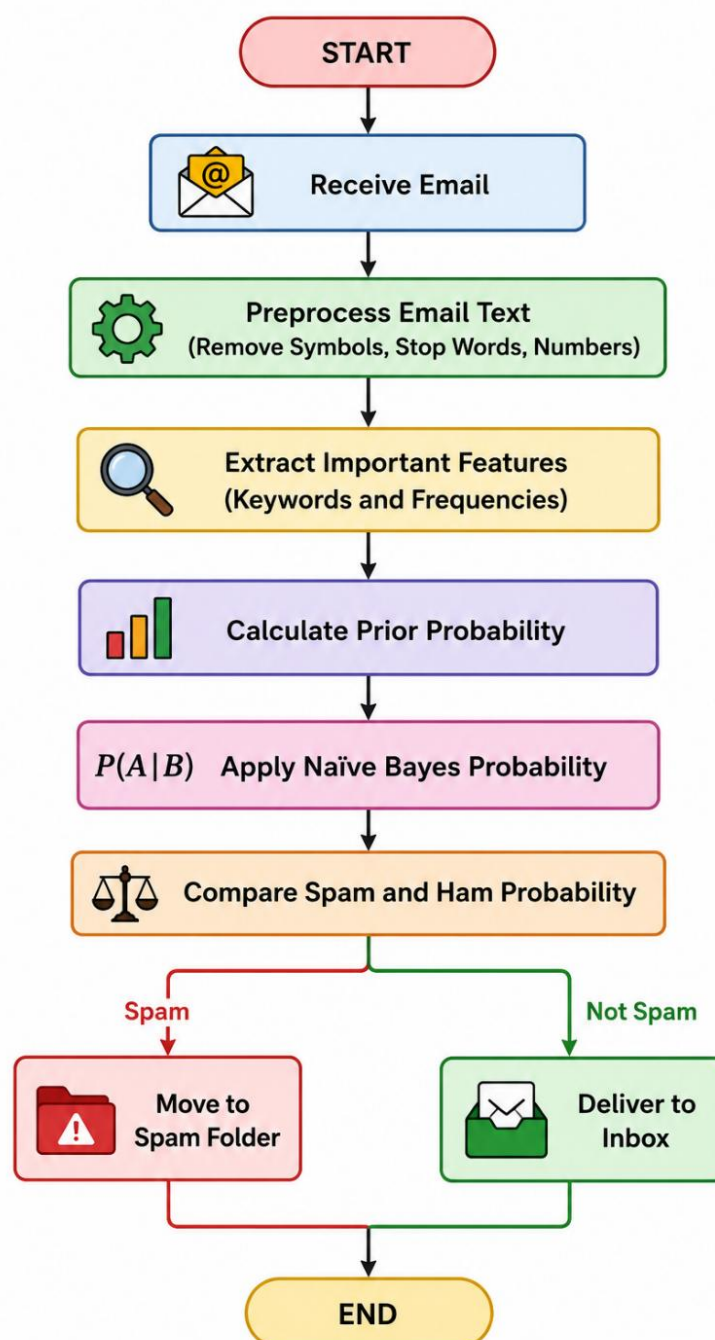
Then classify the email as Spam; otherwise, classify it as Not Spam (Ham).



Working Principle:

1. Collect a dataset containing spam and non-spam emails.
2. Preprocess the emails by removing punctuation, stop words, and unnecessary symbols.
3. Extract important features such as words like Free, Offer, Lottery, Prize, Winner, etc.
4. Calculate the probability of each word appearing in spam and ham emails.
5. Apply Bayes' theorem to compute the posterior probability.
6. Compare the probabilities of the spam and ham classes.
7. If the spam probability is higher, classify the email as Spam; otherwise, classify it as Ham (Not Spam).

Flowchart:



Code:

```
# Using Naive Bayes Classification
# Google Colab - Single Code
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
print("Please choose your Kaggle CSV dataset...")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
print("Uploaded File:", filename)
try:
    df = pd.read_csv(filename, encoding="utf-8")
except:
    df = pd.read_csv(filename, encoding="latin-1")
print("\nDataset Shape:", df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
print("\nFirst 5 Rows:")
display(df.head())
text_column = None
label_column = None
text_names = [
    "text", "message", "email", "body",
    "mail", "content", "v2"
]
label_names = [
    "label", "spam", "category", "class",
    "target", "v1"
]
for col in df.columns:
    if col.lower() in text_names:
        text_column = col
        break
    if col.lower() in label_names:
        label_column = col
        break
if text_column is None or label_column is None:
    print("\nUsing first two columns automatically.")
```

```

label_column = df.columns[0]
text_column = df.columns[1]
print("\nSelected Label Column:", label_column)
print("Selected Text Column :", text_column)
data = df[[text_column, label_column]].copy()
data.columns = ["text", "label"]
data = data.dropna()
data["text"] = data["text"].astype(str)
data["label"] = data["label"].astype(str)
print("\nClean Dataset:")
display(data.head())
def convert_label(value):
    value = value.lower().strip()
    if value == "spam":
        return 1
    elif value == "ham":
        return 0
    elif value == "1":
        return 1
    elif value == "0":
        return 0
    elif value == "true":
        return 1
    elif value == "false":
        return 0
    elif value == "yes":
        return 1
    elif value == "no":
        return 0
    elif value == "not spam":
        return 0
    else:
        return np.nan
data["target"] = data["label"].apply(convert_label)
data = data.dropna(subset=["target"])
data["target"] = data["target"].astype(int)
print("\nClass Distribution:")
print(data["target"].value_counts())
if data["target"].nunique() < 2:
    print("\nERROR: Dataset must contain both Spam and Ham emails.")
else:
    X = data["text"]
    y = data["target"]
    X_train, X_test, y_train, y_test = train_test_split(

```

```

X,
y,
test_size=0.20,
random_state=42,
stratify=y
)
print("\nTraining Samples:", len(X_train))
print("Testing Samples :", len(X_test))
vectorizer = CountVectorizer(
    lowercase=True,
    stop_words="english"
)
X_train_vector = vectorizer.fit_transform(X_train)
X_test_vector = vectorizer.transform(X_test)
print("\nNumber of Features:",
      X_train_vector.shape[1])
model = MultinomialNB()

model.fit(
    X_train_vector,
    y_train
)
y_pred = model.predict(X_test_vector)
accuracy = accuracy_score(
    y_test,
    y_pred
)
print("\n=====")
print("MODEL PERFORMANCE")
print("=====")
print(
    "Accuracy:",
    round(accuracy * 100, 2),
    "%"
)
print("\nClassification Report:")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Ham", "Spam"]
    )
)

```

```

print("\nConfusion Matrix:")

cm = confusion_matrix(
    y_test,
    y_pred
)
print(cm)
probabilities = model.predict_proba(
    X_test_vector
)
results = pd.DataFrame({
    "Email": X_test.values,
    "Actual": y_test.values,

    "Predicted": y_pred,
    "Ham Probability (%)":
        probabilities[:, 0] * 100,
    "Spam Probability (%)":
        probabilities[:, 1] * 100
})
print("\n=====")
print("PROBABILITY DISTRIBUTION")
print("=====")
display(
    results.head(10).round(2)
)
new_email = [
    "Congratulations! You have won a free lottery prize. Click now to claim your reward."
]
new_email_vector = vectorizer.transform(
    new_email
)
new_prediction = model.predict(
    new_email_vector
)
new_probability = model.predict_proba(
    new_email_vector
)
print("\n=====")
print("NEW EMAIL PREDICTION")
print("=====")
if new_prediction[0] == 1:
    print("Prediction: SPAM")
else:

```

```

    print("Prediction: HAM")
print(
    "Ham Probability:",
    round(new_probability[0][0] * 100, 2),
    "%"
)
print(
    "Spam Probability:",
    round(new_probability[0][1] * 100, 2),
    "%"
)
print("\n=====")
print("FINAL RESULT")
print("=====")
print(
    "Naive Bayes probability-based spam",
    "email filtering system implemented successfully."
)
print(
    "Final Accuracy:",
    round(accuracy * 100, 2),
    "%"
)
)

```

Output:

```

... Please choose your Kaggle CSV dataset...
Choose Files email_dataset[1].csv
email_dataset[1].csv(text/csv) - 5629332 bytes, last modified: 8/10/2026 - 100% done
Saving email_dataset[1].csv to email_dataset[1].csv
Uploaded File: email_dataset[1].csv

```

Dataset Shape: (3000, 6)

Column Names:
['from', 'to', 'subject', 'date', 'body', 'label']

First 5 Rows:

	from	to
0	Robert Elz <kre@munnnari.OZ.AU>	Chris Garrigues <cwg-dated-1030377287.06fa6d@D...
1	Steve Burt <Steve_Burt@cursor-system.com>	"zzzzteana@yahoogroups.com" <zzzzteana@yahoo...
2	Tim Chapman <timc@2ubh.com>	zzzzteana <zzzzteana@yahoogroups.com>
3	Monty Solomon <monty@roscom.com>	undisclosed-recipient;
4	Stewart Smith <Stewart.Smith@ee.ed.ac.uk>	zzzzteana@yahoogroups.com

```

*** Class Distribution:
target
0    2500
1     472
Name: count, dtype: int64

Training Samples: 2377
Testing Samples : 595

Number of Features: 36627

=====
MODEL PERFORMANCE
=====
Accuracy: 97.98 %

Classification Report:

```

	precision	recall	f1-score	support
Ham	0.98	1.00	0.99	501
Spam	1.00	0.87	0.93	94
accuracy			0.98	595
macro avg	0.99	0.94	0.96	595
weighted avg	0.98	0.98	0.98	595

```

Confusion Matrix:
[[501  0]
 [ 12 82]]

=====
NEW EMAIL PREDICTION
=====
Prediction: SPAM
Ham Probability: 6.39 %
Spam Probability: 93.61 %

=====
FINAL RESULT
=====
Naive Bayes probability-based spam email filtering system implemented successfully.
Final Accuracy: 97.98 %

```

Advantages:

- Improves diagnostic accuracy by using probabilistic reasoning.
- Handles uncertain, incomplete, and noisy medical data effectively.
- Supports evidence-based clinical decision-making.
- Reduces false positive and false negative diagnoses.
- Helps doctors recommend suitable and personalized treatments.
- Enables early detection of diseases, improving patient outcomes.
- Assists healthcare professionals in prioritizing high-risk patients.
- Reduces diagnostic time and improves hospital efficiency.

Applications:

- Cancer detection and classification.
- Diabetes prediction and risk assessment.
- Heart disease diagnosis.
- COVID-19 diagnosis and severity prediction.
- Medical expert systems.
- Clinical Decision Support Systems (CDSS).
- Medical image analysis (X-rays, MRI, CT scans).
- Predicting infectious diseases and epidemics.
- Personalized medicine and treatment recommendation.

Conclusion:

Bayes Decision Theory is an effective probabilistic approach for disease prediction and medical decision-making. It combines prior medical knowledge with patient symptoms, laboratory test results, and clinical evidence to estimate the probability of disease accurately. By minimizing decision errors and handling uncertainty effectively, it enables healthcare professionals to make faster, more reliable, and evidence-based diagnostic decisions. This approach improves diagnostic accuracy, reduces the chances of misdiagnosis, supports early disease detection, enhances treatment planning, and ultimately contributes to better patient care and improved healthcare outcomes. It is widely used in modern intelligent healthcare systems, medical expert systems, and AI-based clinical decision support applications.

Q2. Illustrate How Bayes Decision Theory Can Be Applied in Medical Diagnosis for Disease Prediction**Definition:**

Bayes Decision Theory is a statistical decision-making approach that uses Bayes' Theorem to determine the probability of a disease based on prior medical knowledge and current patient evidence. It combines the patient's symptoms, medical history, and laboratory test results to estimate the likelihood of disease. This method helps doctors make accurate clinical decisions under uncertain conditions by selecting the diagnosis with the highest probability.

Bayes Decision Theory is widely used in medical diagnosis systems, clinical decision support systems, and artificial intelligence (AI)-based healthcare applications because it improves diagnostic accuracy and reduces the chances of false diagnoses.

Mathematical Formula:

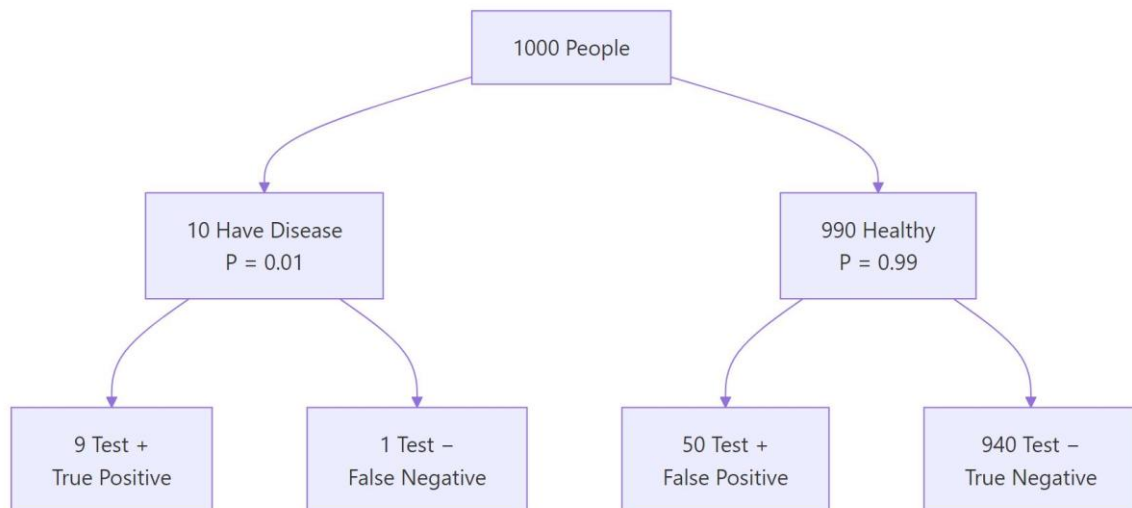
Using **Bayes' Theorem**:

$$P(\text{Disease} | \text{Test}) = \frac{P(\text{Test} | \text{Disease}) \times P(\text{Disease})}{P(\text{Test})}$$

Where:

- **P(Disease | Test)** = Probability that the patient has the disease after the test result.
- **P(Test | Disease)** = Probability of getting a positive test when the disease is actually present.
- **P(Disease)** = Prior probability of the disease.
- **P(Test)** = Overall probability of obtaining the test result.

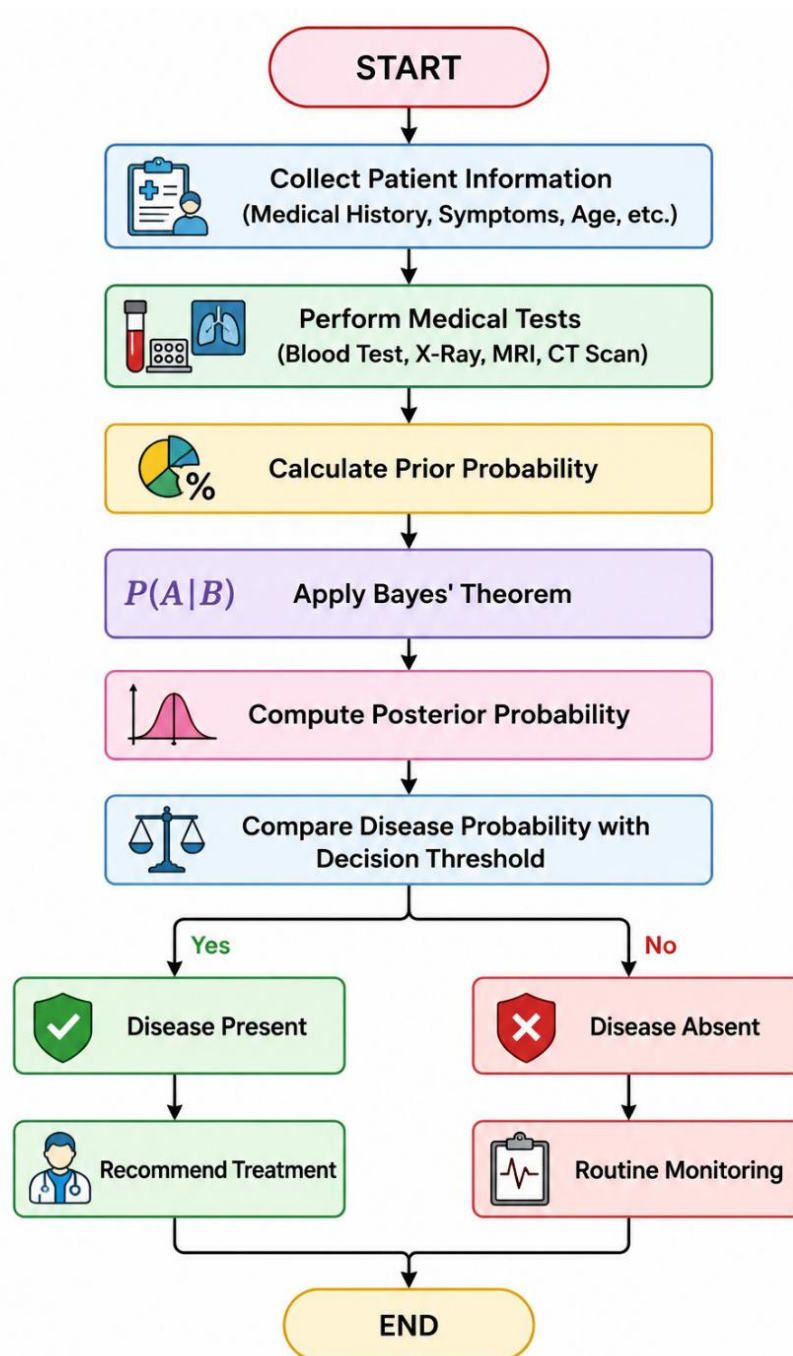
Medical Test — Bayes in Action



Working Principle:

1. Collect the patient's medical history, symptoms, age, and clinical information.
2. Perform laboratory or diagnostic tests such as blood tests, X-rays, CT scans, or MRI scans.
3. Calculate the prior probability of the disease using historical medical records.
4. Apply Bayes' Theorem to combine prior knowledge with the current test results.
5. Compute the posterior probability that the patient has the disease.
6. Compare the posterior probability with a predefined decision threshold.
7. If the disease probability is higher than the threshold, diagnose Disease Present; otherwise, diagnose Disease Absent.

Flowchart:



Code:

```
# STEP 1: IMPORT LIBRARIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
print("Please choose the diabetes.csv file")
uploaded = files.upload()
# Get uploaded filename
filename = list(uploaded.keys())[0]
# Read CSV
data = pd.read_csv(filename)
print("\nFile uploaded successfully!")
print("Filename:", filename) =
print("\nFirst 5 Rows:")
print(data.head())
print("\nDataset Shape:")
print(data.shape)
print("\nColumn Names:")
print(data.columns.tolist())
print("\nMissing Values:")
print(data.isnull().sum())
X = data.drop("Outcome", axis=1)
y = data["Outcome"]
print("\nFeatures:")
print(X.columns.tolist())
print("\nTarget Variable: Outcome")
print("0 = No Diabetes")
print("1 = Diabetes")
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)
=====

```

```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
model = GaussianNB()
# Train model
model.fit(X_train_scaled, y_train)
print("\nGaussian Naive Bayes Model Trained Successfully!")
y_pred = model.predict(X_test_scaled)
# Bayesian probabilities
y_prob = model.predict_proba(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print("\n=====")
print("BAYES DECISION THEORY RESULTS")
print("=====")
print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision :", round(precision * 100, 2), "%")
print("Recall   :", round(recall * 100, 2), "%")
print("F1 Score  :", round(f1 * 100, 2), "%")
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["No Diabetes", "Diabetes"]
    )
)
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Diabetes", "Diabetes"],
    yticklabels=["No Diabetes", "Diabetes"]
)

```

```

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Bayes Decision Theory - Disease Prediction")

plt.show()
print("\n=====")
print("BAYESIAN PROBABILITY RESULTS")
print("=====")
for i in range(10):

    print(
        f"Patient {i+1} : "
        f"No Diabetes = {y_prob[i][0]*100:.2f}% | "
        f"Diabetes = {y_prob[i][1]*100:.2f}% | "
        f"Prediction = {y_pred[i]}"
    )
# New patient:
# Pregnancies = 2
# Glucose = 140
# BloodPressure = 80
# SkinThickness = 35
# Insulin = 120
# BMI = 32.5
# DiabetesPedigreeFunction = 0.45
# Age = 45

new_patient = np.array([
    [2, 140, 80, 35, 120, 32.5, 0.45, 45]
])
# Scale new patient
new_patient_scaled = scaler.transform(new_patient)
# Prediction
prediction = model.predict(new_patient_scaled)
# Probability
probability = model.predict_proba(new_patient_scaled)
print("\n=====")
print("NEW PATIENT PREDICTION")
print("=====")
print(
    "Probability of No Diabetes:",
    round(probability[0][0] * 100, 2),
    "%"
)

```

```

print(
    "Probability of Diabetes:",
    round(probability[0][1] * 100, 2),
    "%")
)
if prediction[0] == 1:
    print("\nFinal Prediction: DIABETES")
else:
    print("\nFinal Prediction: NO DIABETES")

```

Output:

```

Choose Files archive (3).zip
... archive (3).zip(application/x-zip-compressed) - 9353 bytes, last modified: 8/11/2026 - 100% done
Saving archive (3).zip to archive (3).zip

Dataset loaded successfully!

First 5 Rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin  BMI  \
0             6     148             72.0             35    169.5  33.6
1             1      85             66.0             29    102.5  26.6
2             8     183             64.0             32    169.5  23.3
3             1      89             66.0             23     94.0  28.1
4             0     137             40.0             35    168.0  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                0.627     50         1
1                0.351     31         0
2                0.672     32         1
3                0.167     21         0
4                2.288     33         1

Dataset Shape:
(768, 9)

Column Names:
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome']

Missing Values:
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0

Insulin           0
... BMI           0
DiabetesPedigreeFunction  0
Age               0
Outcome           0
dtype: int64

Features:
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']

Target: Outcome
0 = No Diabetes
1 = Diabetes

Training Samples: 614
Testing Samples: 154

Gaussian Naive Bayes Model Trained Successfully!

=====
BAYES DECISION THEORY RESULTS
=====
Accuracy   : 72.73 %
Precision  : 60.34 %
Recall     : 64.81 %
F1 Score   : 62.5 %

```

```

Classification Report:
              precision    recall  f1-score   support

   No Diabetes         0.80      0.77      0.79        100
    Diabetes         0.60      0.65      0.62         54

   accuracy              0.73          154
  macro avg         0.70      0.71      0.71        154
 weighted avg         0.73      0.73      0.73        154


Confusion Matrix:
[[77 23]
 [19 35]]

...
=====
BAYESIAN DISEASE PROBABILITIES
=====
Patient 1: No Diabetes = 60.54% | Diabetes = 39.46% | Prediction = 0
Patient 2: No Diabetes = 78.91% | Diabetes = 21.09% | Prediction = 0
Patient 3: No Diabetes = 76.72% | Diabetes = 23.28% | Prediction = 0
Patient 4: No Diabetes = 81.47% | Diabetes = 18.53% | Prediction = 0
Patient 5: No Diabetes = 99.39% | Diabetes = 0.61% | Prediction = 0
Patient 6: No Diabetes = 97.27% | Diabetes = 2.73% | Prediction = 0
Patient 7: No Diabetes = 14.42% | Diabetes = 85.58% | Prediction = 1
Patient 8: No Diabetes = 0.25% | Diabetes = 99.75% | Prediction = 1
Patient 9: No Diabetes = 99.38% | Diabetes = 0.62% | Prediction = 0
Patient 10: No Diabetes = 0.54% | Diabetes = 99.46% | Prediction = 1

=====
NEW PATIENT MEDICAL PREDICTION
=====
Probability of No Diabetes: 54.33 %
Probability of Diabetes: 45.67 %

Final Prediction: NO DIABETES

```

Advantages:

- Improves the accuracy of disease prediction.
- Handles uncertain and incomplete medical information.
- Supports evidence-based clinical decision making.
- Reduces false positive and false negative diagnoses.
- Helps doctors recommend appropriate treatment plans.
- Can be integrated into AI-based healthcare systems.

Applications:

- Cancer diagnosis
- Heart disease prediction
- Diabetes detection
- COVID-19 diagnosis
- Medical expert systems

- Clinical Decision Support Systems (CDSS)
- AI-based healthcare applications

Conclusion:

Bayes Decision Theory is an effective probabilistic approach for medical diagnosis. By combining prior medical knowledge with current patient test results, it accurately predicts the likelihood of disease. This approach assists healthcare professionals in making faster, more reliable, and evidence-based diagnostic decisions, ultimately improving patient care and treatment outcomes.

Q3. Model the Outcome of a Weather Forecasting Problem Using Random Variables and Probability Theory

Definition:

A random variable is a variable whose value depends on the outcome of a random event. In weather forecasting, the weather condition (Rain or No Rain) is uncertain, so it can be represented using probability theory.

Probability Theory is a mathematical method used to measure the likelihood of different weather events occurring.

Mathematical Formula:

Probability of an Event

$$P(E) = \frac{\text{Number of Favorable Outcomes}}{\text{Total Number of Outcomes}}$$

Expected Value

$$E(X) = \sum X \times P(X)$$

For this example,

$$E(X) = 1(0.70) + 0(0.30) = 0.70$$

This means there is a 70% chance of rainfall.

Example

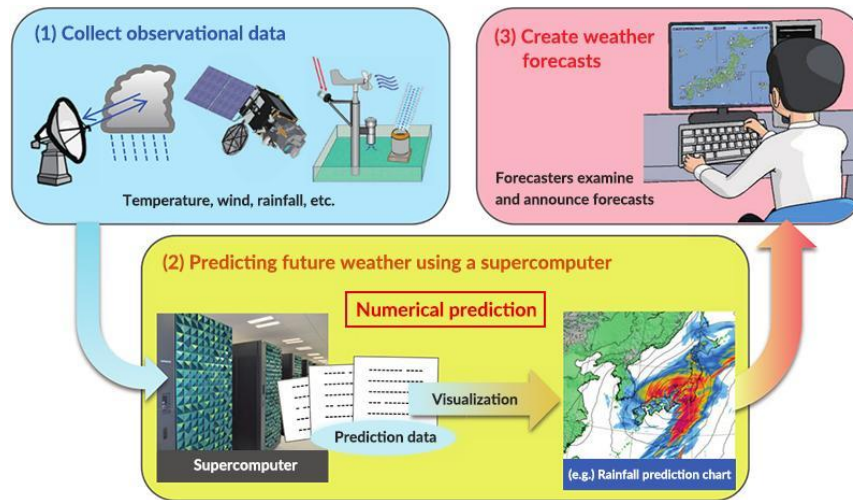
Suppose meteorological data predicts:

- Rain = 70%
- No Rain = 30%

Since

$$0.70 > 0.30$$

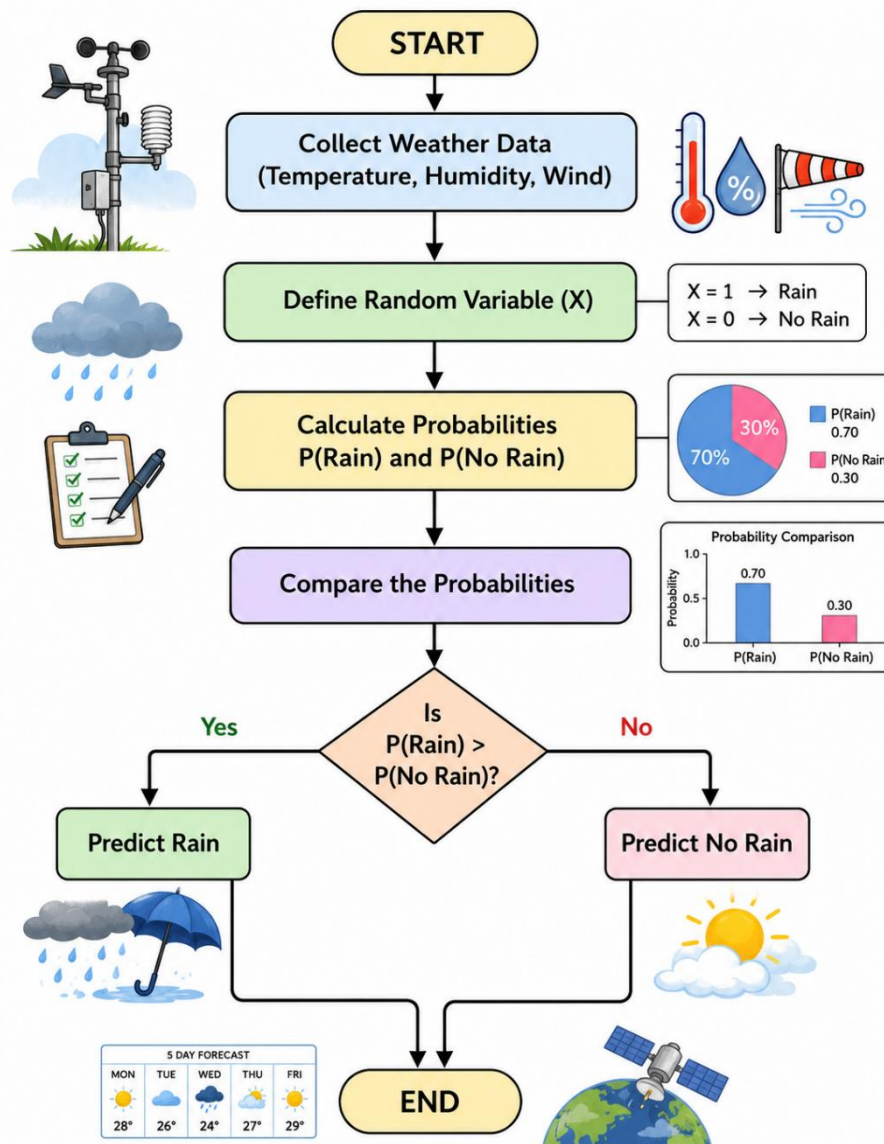
the forecast predicts Rain.



Explanation:

- Weather observations such as humidity, temperature, wind speed, and atmospheric pressure are collected.
- These values are treated as inputs.
- A probability model estimates the likelihood of rain.
- The event having the highest probability is selected as the weather forecast.
- Probability theory helps improve forecasting accuracy under uncertain conditions.

Flowchart:



Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files
print("Please upload weatherAUS.csv")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
data = pd.read_csv(filename)
print("\nDataset uploaded successfully!")
print("Filename:", filename)
print("\nFirst 5 Rows:")
print(data.head())

print("\nDataset Shape:")
print(data.shape)

print("\nColumn Names:")
print(data.columns.tolist)
# We use RainToday to predict RainTomorrow
weather = data[
    ["RainToday", "RainTomorrow", "Humidity3pm", "Pressure3pm",
     "Temp3pm", "WindSpeed3pm"]
].copy()
# Remove missing values
weather = weather.dropna()
print("\nCleaned Dataset:")
print(weather.head())

print("\nNumber of Records:", len(weather))
# X = RainTomorrow
#
# X = 1 -> Rain
# X = 0 -> No Rain
weather["RainTomorrow_Binary"] = (
    weather["RainTomorrow"] == "Yes"
).astype(int)
total = len(weather)
rain_count = weather["RainTomorrow_Binary"].sum()

no_rain_count = total - rain_count

P_rain = rain_count / total
```

```

P_no_rain = no_rain_count / total
print("\n=====")
print("PROBABILITY OF WEATHER OUTCOME")
print("=====")

print("Total observations:", total)

print(
    "P(Rain Tomorrow) =",
    round(P_rain, 4),
    "=",
    round(P_rain * 100, 2),
    "%"
)
print(
    "P(No Rain Tomorrow) =",
    round(P_no_rain, 4),
    "=",
    round(P_no_rain * 100, 2),
    "%"
)
# Probability of Rain Tomorrow
# when it rained Today

rain_today_data = weather[
    weather["RainToday"] == "Yes"
]

rain_tomorrow_given_today_rain = (
    rain_today_data["RainTomorrow_Binary"].mean()
)
print("\n=====")
print("CONDITIONAL PROBABILITY")
print("=====")

print(
    "P(Rain Tomorrow | Rain Today) =",
    round(rain_tomorrow_given_today_rain, 4)
)

print(
    "Percentage =",
    round(rain_tomorrow_given_today_rain * 100, 2),
    "%"
)

```

```

)

# Probability of Rain Tomorrow
# when it did NOT rain Today

no_rain_today_data = weather[
    weather["RainToday"] == "No"
]

rain_tomorrow_given_no_rain_today = (
    no_rain_today_data["RainTomorrow_Binary"].mean()
)

print(
    "P(Rain Tomorrow | No Rain Today) =",
    round(rain_tomorrow_given_no_rain_today, 4)
)

print(
    "Percentage =",
    round(rain_tomorrow_given_no_rain_today * 100, 2),
    "%"
)

probability_table = pd.DataFrame({
    "Weather Outcome": [
        "No Rain",
        "Rain"
    ],
    "Probability": [
        P_no_rain,
        P_rain
    ]
})

print("\n=====")
print("RANDOM VARIABLE PROBABILITY TABLE")
print("=====")

print(probability_table)
plt.figure(figsize=(7, 5))

plt.bar(
    probability_table["Weather Outcome"],
    probability_table["Probability"]

```

```

)

plt.xlabel("Weather Outcome")
plt.ylabel("Probability")

plt.title(
    "Probability Distribution of Rain Tomorrow"
)

plt.ylim(0, 1)

plt.show()
print("\n=====")
print("WEATHER FORECAST")
print("=====")

if P_rain > P_no_rain:
    print("Forecast: Rain is more probable tomorrow.")
else:
    print("Forecast: No Rain is more probable tomorrow.")
np.random.seed(42)

simulation = np.random.choice(
    ["Rain", "No Rain"],
    size=1000,
    p=[P_rain, P_no_rain]
)

simulation_probability = (
    np.mean(simulation == "Rain")
)

print("\n=====")
print("RANDOM SIMULATION")
print("=====")

print(
    "Simulated P(Rain) =",
    round(simulation_probability, 4)
)

print(
    "Simulated P(No Rain) =",
    round(1 - simulation_probability, 4)
)

```

```

)

print("\n=====")
print("FINAL RESULT")
print("=====")

print(
    "Probability of Rain Tomorrow:",
    round(P_rain * 100, 2),
    "%"
)

print(
    "Probability of No Rain Tomorrow:",
    round(P_no_rain * 100, 2),
    "%"
)

print(
    "P(Rain Tomorrow | Rain Today):",
    round(
        rain_tomorrow_given_today_rain * 100,
        2
    ),
    "%"
)

print(
    "P(Rain Tomorrow | No Rain Today):",
    round(
        rain_tomorrow_given_no_rain_today * 100,
        2
    ),
    "%"
)

print("\nWeather forecasting model completed!")

```

Output:

```
... Please upload weatherAUS.csv
Choose Files weatherAUS[1].csv
weatherAUS[1].csv(text/csv) - 14094055 bytes, last modified: 8/11/2026 - 100% done
Saving weatherAUS[1].csv to weatherAUS[1].csv

Dataset uploaded successfully!
Filename: weatherAUS[1].csv

First 5 Rows:
      Date Location  MinTemp  MaxTemp  Rainfall  Evaporation  Sunshine \
0  2008-12-01  Albury    13.4    22.9      0.6         NaN       NaN
1  2008-12-02  Albury     7.4    25.1      0.0         NaN       NaN
2  2008-12-03  Albury    12.9    25.7      0.0         NaN       NaN
3  2008-12-04  Albury     9.2    28.0      0.0         NaN       NaN
4  2008-12-05  Albury    17.5    32.3      1.0         NaN       NaN

      WindGustDir  WindGustSpeed  WindDir9am  ... Humidity9am  Humidity3pm \
0              W             44.0           W  ...      71.0       22.0
1            WNW             44.0          NNW  ...      44.0       25.0
2            WSW             46.0           W  ...      38.0       30.0
3             NE             24.0          SE  ...      45.0       16.0
4              W             41.0          ENE  ...      82.0       33.0

      Pressure9am  Pressure3pm  Cloud9am  Cloud3pm  Temp9am  Temp3pm  RainToday \
0         1007.7        1007.1        8.0      NaN     16.9     21.8       No
1         1010.6        1007.8       NaN      NaN     17.2     24.3       No
2         1007.6        1008.7       NaN      2.0     21.0     23.2       No
3         1017.6        1012.8       NaN      NaN     18.1     26.5       No
4         1010.8        1006.0        7.0      8.0     17.8     29.7       No

      RainTomorrow
0              No
1              No
2              No
3              No
4              No

[5 rows x 23 columns]

Dataset Shape:
(145460, 23)

Column Names:
['Date', 'Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation', 'Sunshine',

Cleaned Dataset:
      RainToday  RainTomorrow  Humidity3pm  Pressure3pm  Temp3pm  WindSpeed3pm
0           No           No         22.0      1007.1     21.8         24.0
1           No           No         25.0      1007.8     24.3         22.0
2           No           No         30.0      1008.7     23.2         26.0
3           No           No         16.0      1012.8     26.5          9.0
4           No           No         33.0      1006.0     29.7         20.0

Number of Records: 124586

=====
PROBABILITY OF WEATHER OUTCOME
=====
Total observations: 124586
P(Rain Tomorrow) = 0.2211 = 22.11 %
P(No Rain Tomorrow) = 0.7789 = 77.89 %

=====
CONDITIONAL PROBABILITY
=====
P(Rain Tomorrow | Rain Today) = 0.4643
Percentage = 46.43 %
P(Rain Tomorrow | No Rain Today) = 0.1513
Percentage = 15.13 %

=====
RANDOM VARIABLE PROBABILITY TABLE
=====
      Weather Outcome  Probability
0           No Rain         0.7789
1            Rain          0.2211
```



```

=====
WEATHER FORECAST
=====
Forecast: No Rain is more probable tomorrow.

=====
RANDOM SIMULATION
=====
Simulated P(Rain) = 0.238
Simulated P(No Rain) = 0.762

=====
FINAL RESULT
=====
Probability of Rain Tomorrow: 22.11 %
Probability of No Rain Tomorrow: 77.89 %
P(Rain Tomorrow | Rain Today): 46.43 %
P(Rain Tomorrow | No Rain Today): 15.13 %

Weather forecasting model completed!

```

Applications :

1. **Weather Prediction** – Predicts the probability of rain, sunshine, storms, or snowfall.
2. **Agriculture** – Helps farmers plan irrigation, planting, and harvesting activities.
3. **Disaster Management** – Provides early warnings for cyclones, floods, and heavy rainfall.
4. **Aviation** – Assists pilots in planning safe flight routes based on weather conditions.
5. **Marine Navigation** – Helps ships avoid rough seas and storms.

Advantages:

1. Improves Forecast Accuracy by using mathematical probability models.
2. Supports Better Decision Making in agriculture, aviation, and disaster management.
3. Reduces Risk by providing early warnings of severe weather conditions.
4. Handles Uncertainty by estimating the likelihood of different weather events.
5. Efficient Resource Planning for water management, energy production, and transportation.
6. Easy to Interpret through probability values (e.g., 80% chance of rain).
7. Cost Effective by reducing losses caused by unexpected weather changes.
8. Widely Applicable in environmental monitoring, climate studies, and public safety.

Conclusion:

Random variables and probability theory provide a systematic and mathematical approach to weather forecasting by estimating the likelihood of different weather conditions such as rain or no rain. By analyzing weather data and calculating probabilities, meteorologists can make more accurate and reliable predictions. This approach helps improve decision-making in agriculture, disaster management, transportation, aviation, and other weather-dependent sectors, making weather forecasting more effective and dependable.

Q4. Apply conditional probability concepts to design a fraud detection model for suspicious banking transactions.**Definition:**

Conditional Probability is the probability of an event occurring given that another related event has already occurred. It helps determine how the occurrence of one event affects the likelihood of another event. In probability theory, conditional probability is widely used to analyze dependent events and make predictions based on available information.

In banking fraud detection, conditional probability is used to estimate the probability that a transaction is fraudulent when suspicious behavior has already been detected. Such suspicious activities may include unusually large transaction amounts, transactions from unfamiliar locations, multiple failed login attempts, abnormal transaction times, or frequent transfers within a short period. By analyzing these conditions, banks can calculate the probability of fraud and decide whether to approve, block, or investigate the transaction. This approach improves fraud detection accuracy, reduces financial losses, and enhances the security of digital banking systems.

Mathematical formula:

Let:

- **F** = Transaction is Fraudulent
- **S** = Transaction is Suspicious

The probability that a suspicious transaction is fraudulent is:

$$P(F|S) = \frac{P(F \cap S)}{P(S)}$$

Where:

- $P(F | S)$ = Probability that the transaction is fraudulent given it is suspicious.
- $P(F \cap S)$ = Probability that the transaction is both fraudulent and suspicious.

- $P(S)$ = Probability of a suspicious transaction.

Example

Suppose:

- Probability of suspicious transactions =
 $P(S) = 0.20$
- Probability of fraud and suspicious transaction =
 $P(F \cap S) = 0.16$

Then,

$$P(F | S) = \frac{0.16}{0.20} = 0.80$$

Since

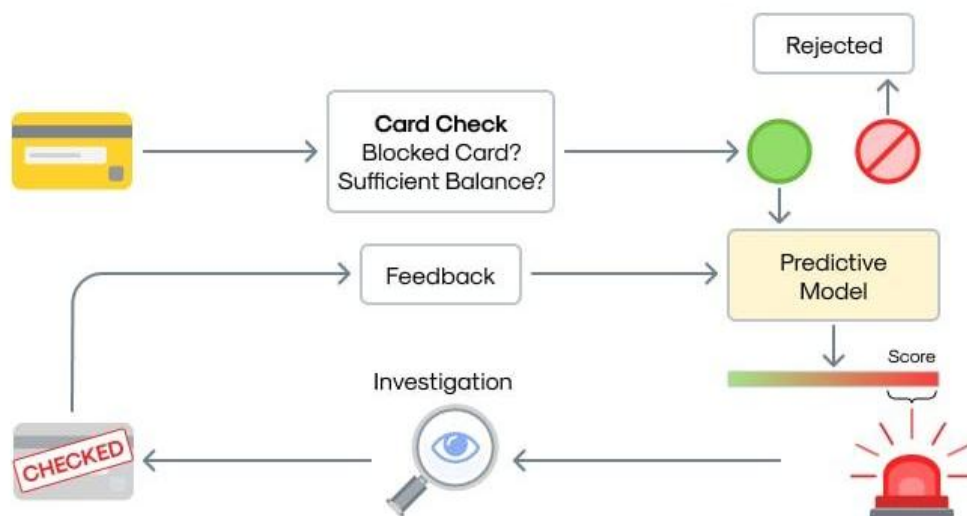
$$P(F | S) = 0.80$$

there is an 80% probability that the suspicious transaction is fraudulent.

Explanation:

- The banking system collects transaction details.
- Suspicious activities such as unusual transaction amount, foreign location, multiple failed login attempts, or unusual transaction time are detected.
- Conditional probability is calculated.
- If the fraud probability exceeds the threshold (e.g., 70%), the transaction is marked as Fraud.
- Otherwise, it is treated as a Normal Transaction.

Credit Card Fraud Prevention Techniques



Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score
)
print("Please upload creditcard.csv")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
data = pd.read_csv(filename)
print("\nFile uploaded successfully!")
print("Filename:", filename)
print("\nFirst 5 Rows:")
print(data.head())
print("\nDataset Shape:")
print(data.shape)
print("\nColumn Names:")
print(data.columns.tolist())
print("\nClass Distribution:")
print(data["Class"].value_counts())
print("\nClass Percentage:")
print(data["Class"].value_counts(normalize=True) * 100)
print("\nMissing Values:")
print(data.isnull().sum().sum())
X = data.drop("Class", axis=1)
y = data["Class"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
```

```

        stratify=y
    )

    print("\nTraining Samples:", len(X_train))
    print("Testing Samples:", len(X_test))
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    model = GaussianNB()
    model.fit(X_train_scaled, y_train)
    print("\nNaive Bayes Fraud Detection Model Trained!")
    y_pred = model.predict(X_test_scaled)
    y_probability = model.predict_proba(X_test_scaled)
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(
        y_test,
        y_pred,
        zero_division=0
    )
    recall = recall_score(
        y_test,
        y_pred,
        zero_division=0
    )

    f1 = f1_score(
        y_test,
        y_pred,
        zero_division=0
    )

    auc = roc_auc_score(
        y_test,
        y_probability[:, 1]
    )

    print("\n=====")
    print("FRAUD DETECTION RESULTS")
    print("=====")
    print("Accuracy :", round(accuracy * 100, 2), "%")
    print("Precision :", round(precision * 100, 2), "%")
    print("Recall   :", round(recall * 100, 2), "%")
    print("F1 Score  :", round(f1 * 100, 2), "%")
    print("ROC-AUC   :", round(auc, 4))
    print("\nClassification Report:")

```

```

print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Genuine", "Fraud"],
        zero_division=0
    )
)
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Genuine", "Fraud"],
    yticklabels=["Genuine", "Fraud"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Fraud Detection - Confusion Matrix")
plt.show()
print("\n=====")
print("CONDITIONAL PROBABILITY RESULTS")
print("=====")
for i in range(10):

    print(
        f"Transaction {i+1}: "
        f"P(Genuine | Features) = "
        f"{y_probability[i][0] * 100:.2f}% | "
        f"P(Fraud | Features) = "
        f"{y_probability[i][1] * 100:.2f}% | "
        f"Prediction = "
        f'{ "FRAUD" if y_pred[i] == 1 else "GENUINE" }'
    )
P_fraud = (y == 1).mean()
P_genuine = (y == 0).mean()
print("\n=====")
print("PRIOR PROBABILITIES")
print("=====")

```

```

print(
    "P(Fraud) =",
    round(P_fraud * 100, 4),
    "%"
)

print(
    "P(Genuine) =",
    round(P_genuine * 100, 4),
    "%"
)

new_transaction = X_test.iloc[[0]]
new_transaction_scaled = scaler.transform(
    new_transaction
)

new_prediction = model.predict(
    new_transaction_scaled
)

new_probability = model.predict_proba(
    new_transaction_scaled
)

print("\n=====")
print("NEW TRANSACTION PREDICTION")
print("=====")

print(
    "P(Genuine | Transaction) =",
    round(new_probability[0][0] * 100, 2),
    "%"
)

print(
    "P(Fraud | Transaction) =",
    round(new_probability[0][1] * 100, 2),
    "%"
)

if new_prediction[0] == 1:
    print("\nFinal Decision: SUSPICIOUS / FRAUD")
else:
    print("\nFinal Decision: GENUINE")

```

Output:

```
... Please upload creditcard.csv
Choose Files creditcard[1].csv
creditcard[1].csv(text/csv) - 150828752 bytes, last modified: 8/11/2026 - 100% done
Saving creditcard[1].csv to creditcard[1].csv

File uploaded successfully!
Filename: creditcard[1].csv

First 5 Rows:
  Time      V1      V2      V3      V4      V5      V6      V7 \
0  0.0 -1.359807 -0.072781 2.536347 1.378155 -0.338321 0.462388 0.239599
1  0.0  1.191857 0.266151 0.166480 0.448154 0.060018 -0.082361 -0.078803
2  1.0 -1.358354 -1.340163 1.773209 0.379780 -0.503198 1.800499 0.791461
3  1.0 -0.966272 -0.185226 1.792993 -0.863291 -0.010309 1.247203 0.237609
4  2.0 -1.158233 0.877737 1.548718 0.403034 -0.407193 0.095921 0.592941

      V8      V9      ...      V21      V22      V23      V24      V25 \
0  0.098698 0.363787 ... -0.018307 0.277838 -0.110474 0.066928 0.128539
1  0.085102 -0.255425 ... -0.225775 -0.638672 0.101288 -0.339846 0.167170
2  0.247676 -1.514654 ... 0.247998 0.771679 0.909412 -0.689281 -0.327642
3  0.377436 -1.387024 ... -0.108300 0.005274 -0.190321 -1.175575 0.647376
4 -0.270533 0.817739 ... -0.009431 0.798278 -0.137458 0.141267 -0.206010

      V26      V27      V28  Amount  Class
0 -0.189115 0.133558 -0.021053 149.62    0
1  0.125895 -0.008983 0.014724  2.69    0
2 -0.139097 -0.055353 -0.059752 378.66    0
3 -0.221929 0.062723 0.061458 123.50    0
4  0.502292 0.219422 0.215153  69.99    0

[5 rows x 31 columns]

Dataset Shape:
(284807, 31)

Column Names:
['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10', 'V11', 'V12',

Class Distribution:
Class
0    284315
1         492
Name: count, dtype: int64

Class Percentage:
Class
0    99.827251
1     0.172749
Name: proportion, dtype: float64

Missing Values:
0

Training Samples: 227845
Testing Samples: 56962

Naive Bayes Fraud Detection Model Trained!

=====
FRAUD DETECTION RESULTS
=====
Accuracy : 97.64 %
Precision : 5.88 %
Recall : 84.69 %
F1 Score : 10.99 %
ROC-AUC : 0.9632

Classification Report:
      precision    recall  f1-score   support

   Genuine      1.00      0.98      0.99     56864
    Fraud       0.06      0.85      0.11         98

 accuracy      0.98     56962
 macro avg       0.53      0.91      0.55     56962
 weighted avg       1.00      0.98      0.99     56962

Confusion Matrix:
[[55535 1329]
 [ 15 83]]
```



```

*** =====
CONDITIONAL PROBABILITY RESULTS
=====
Transaction 1: P(Genuine | Features) = 100.00% | P(Fraud | Features) = 0.00% |
Transaction 2: P(Genuine | Features) = 100.00% | P(Fraud | Features) = 0.00% |
Transaction 3: P(Genuine | Features) = 3.81% | P(Fraud | Features) = 96.19% |
Transaction 4: P(Genuine | Features) = 100.00% | P(Fraud | Features) = 0.00% |
Transaction 5: P(Genuine | Features) = 100.00% | P(Fraud | Features) = 0.00% |
Transaction 6: P(Genuine | Features) = 100.00% | P(Fraud | Features) = 0.00% |
Transaction 7: P(Genuine | Features) = 100.00% | P(Fraud | Features) = 0.00% |
Transaction 8: P(Genuine | Features) = 100.00% | P(Fraud | Features) = 0.00% |
Transaction 9: P(Genuine | Features) = 100.00% | P(Fraud | Features) = 0.00% |
Transaction 10: P(Genuine | Features) = 100.00% | P(Fraud | Features) = 0.00%

=====
PRIOR PROBABILITIES
=====
P(Fraud) = 0.1727 %
P(Genuine) = 99.8273 %

=====
NEW TRANSACTION PREDICTION
=====
P(Genuine | Transaction) = 100.0 %
P(Fraud | Transaction) = 0.0 %

```

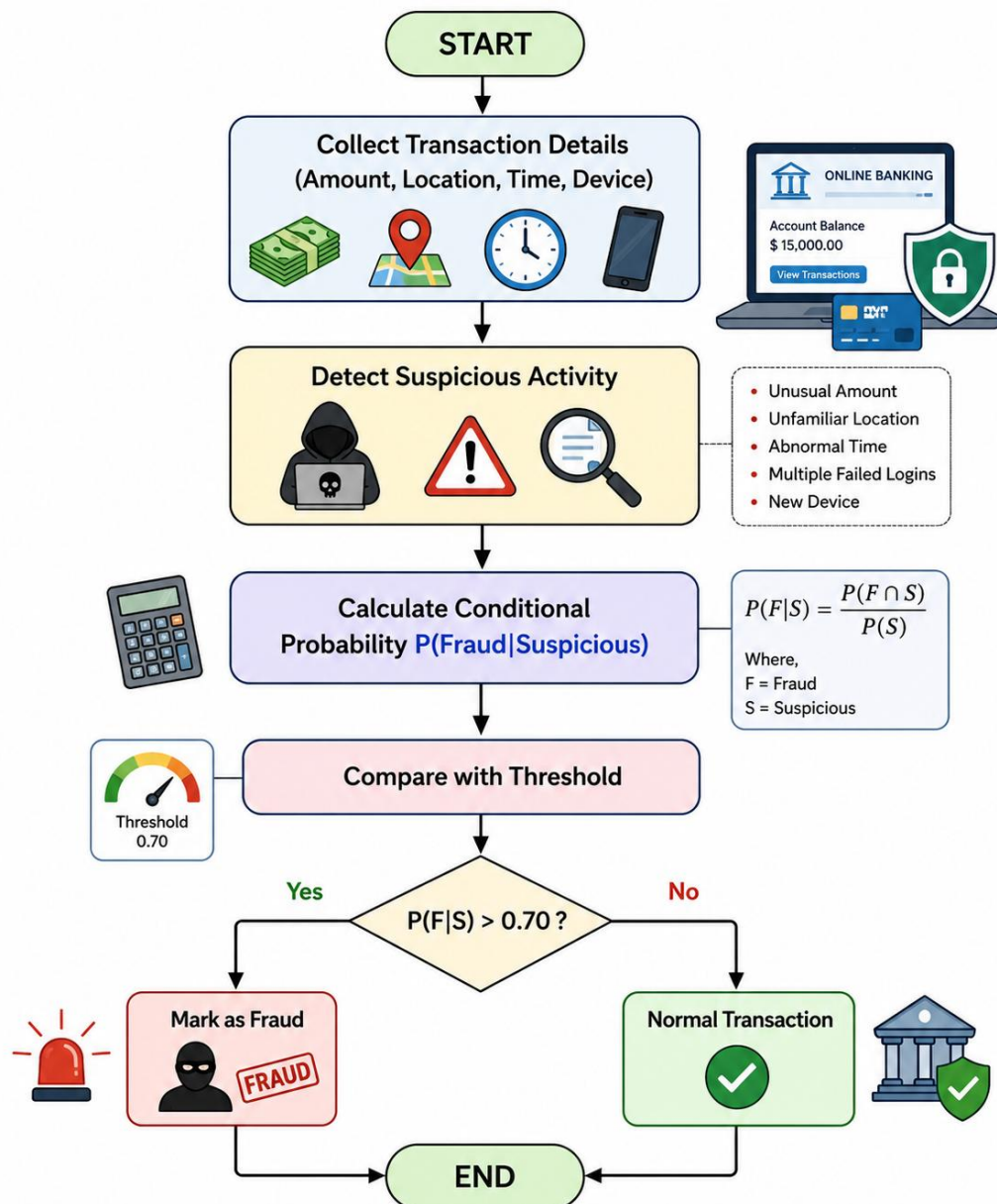
Advantages:

1. Detects fraudulent transactions quickly.
2. Improves banking security.
3. Reduces financial losses.
4. Supports real-time fraud detection.
5. Improves customer trust.
6. Helps banks make accurate decisions.
7. Minimizes false fraud alerts.
8. Easy to implement using probability models.

Applications

1. Online banking fraud detection.
2. Credit card fraud monitoring.
3. ATM transaction security.
4. Internet banking authentication.
5. Digital payment fraud prevention.
6. Financial risk management.
7. Anti-money laundering (AML) systems.
8. Mobile banking security.

Flowchart:



Conclusion:

Conditional probability provides an effective approach for detecting fraudulent banking transactions by calculating the likelihood of fraud based on suspicious activities. By analyzing factors such as transaction amount, location, time, and device information, banks can identify high-risk transactions more accurately. This probability-based model enables real-time fraud detection, reduces financial losses, enhances customer security, and supports reliable decision-making in modern banking systems.

Q5. Construct a Machine Learning Workflow to Classify Handwritten Digits by Applying the Stages of a Learning System

Definition:

A Machine Learning (ML) workflow is a sequence of steps used to train a computer to recognize patterns and make predictions from data. In handwritten digit classification, the workflow enables the system to identify digits (0–9) by learning from labeled handwritten images. The learning system processes the images, extracts useful features, trains a machine learning model, evaluates its performance, and predicts the correct digit for new handwritten inputs. This workflow improves classification accuracy and automates digit recognition in applications such as postal mail sorting, bank cheque processing, and document digitization.

Mathematical formula:

Let:

- X = Input handwritten digit image
- Y = Predicted digit (0–9)
- $f(X)$ = Machine learning model

The prediction model is:

$$Y = f(X)$$

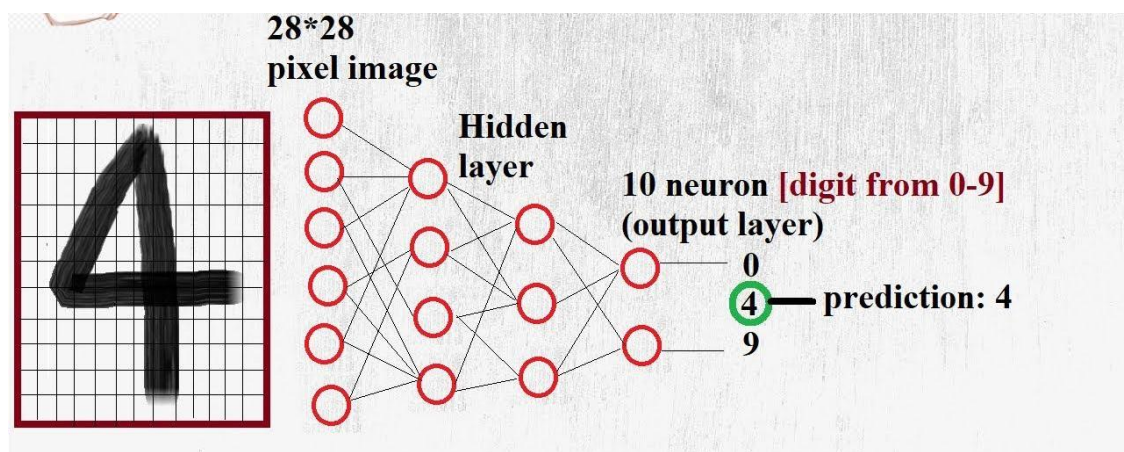
Where:

- X = Input image
- $f(X)$ = Trained machine learning classifier
- Y = Predicted handwritten digit

Example:

Suppose the input image contains the handwritten digit:

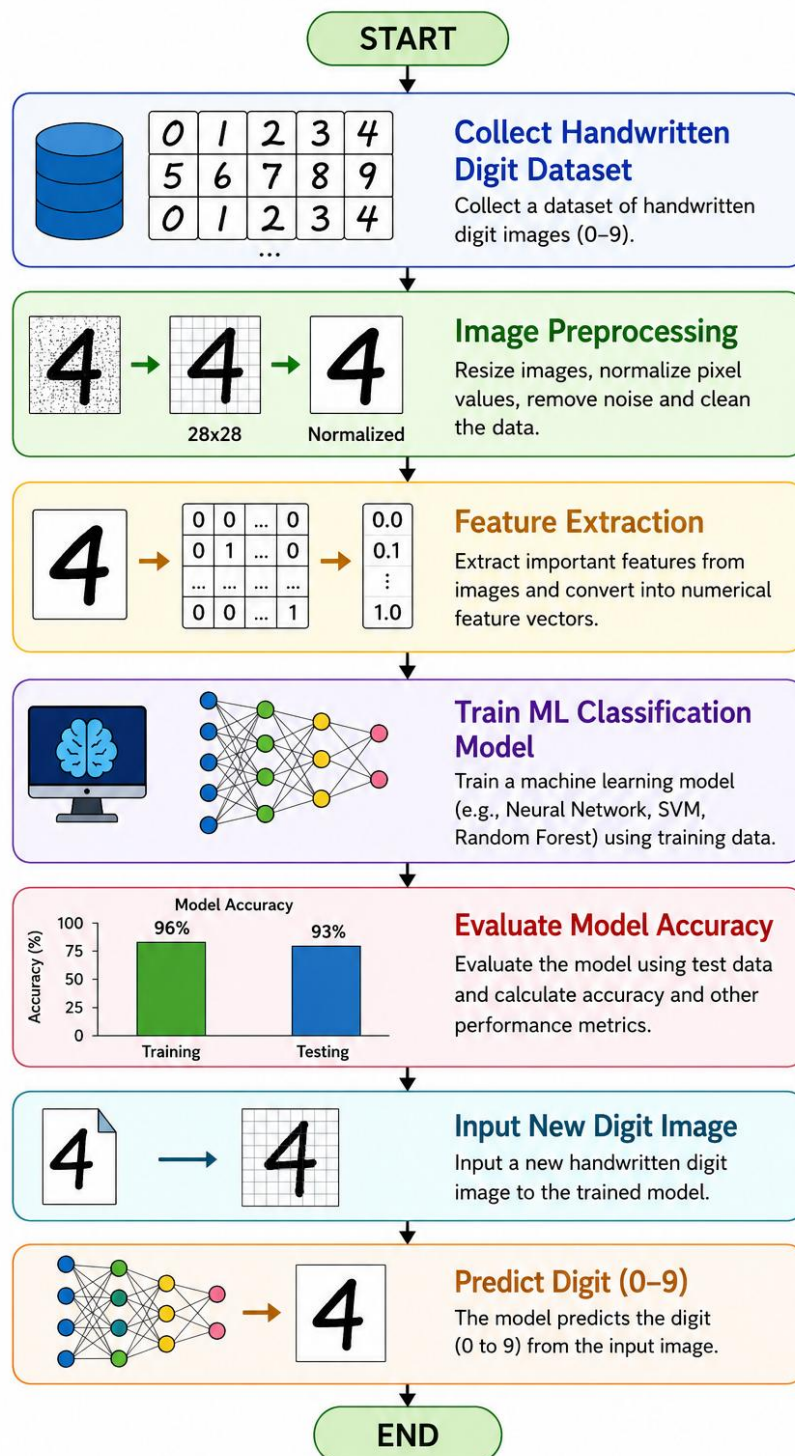
- X = Image of digit '4'
- After preprocessing and classification,
- $Y = f(X) = 4$
- The model correctly predicts the digit as 4



Explanation:

- Collect a dataset of handwritten digit images.
- Preprocess the images by resizing and normalizing pixel values.
- Extract important image features.
- Split the dataset into training and testing sets.
- Train a machine learning classifier using the training data.
- Evaluate the model using test data.
- Predict the digit for a new handwritten image.

Flowchart:



Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)
print("Please upload mnist_train.csv")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
data = pd.read_csv(filename)
print("\nFile uploaded successfully!")
print("Filename:", filename)
print("\nFirst 5 Rows:")
print(data.head())
print("\nDataset Shape:")
print(data.shape)
print("\nColumn Names:")
print(data.columns[:10].tolist(), "...")
# 'label' contains the actual digit
y = data["label"]
# All remaining columns are pixel features
X = data.drop("label", axis=1)
print("\nNumber of Features:", X.shape[1])
print("Number of Classes:", y.nunique())
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    image = X.iloc[i].values.reshape(28, 28)
    plt.imshow(image, cmap="gray")
    plt.title("Digit: " + str(y.iloc[i]))
    plt.axis("off")
plt.tight_layout()
plt.show()
X = X / 255.0
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```

    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)
print("\nTraining Random Forest Model...")
model.fit(X_train, y_train)
print("Model Training Completed!")
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("\n=====")
print("HANDWRITTEN DIGIT CLASSIFICATION")
print("=====")

print(
    "Test Accuracy:",
    round(accuracy * 100, 2),
    "%"
)
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred
    )
)
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)
plt.figure(figsize=(12, 6))
for i in range(15):
    plt.subplot(3, 5, i + 1)
    image = X_test.iloc[i].values.reshape(28, 28)
    plt.imshow(image, cmap="gray")

plt.title(

```

```

        "Actual: " + str(y_test.iloc[i]) +
        "\nPredicted: " + str(y_pred[i])
    )
    plt.axis("off")
plt.tight_layout()
plt.show()
sample_index = 0
sample = X_test.iloc[[sample_index]]
prediction = model.predict(sample)
print("\n=====")
print("SINGLE DIGIT PREDICTION")
print("=====")
print("Actual Digit  :", y_test.iloc[sample_index])
print("Predicted Digit :", prediction[0])

```

Output:

... Upload mnist_train.csv

mnist_train[1].csv

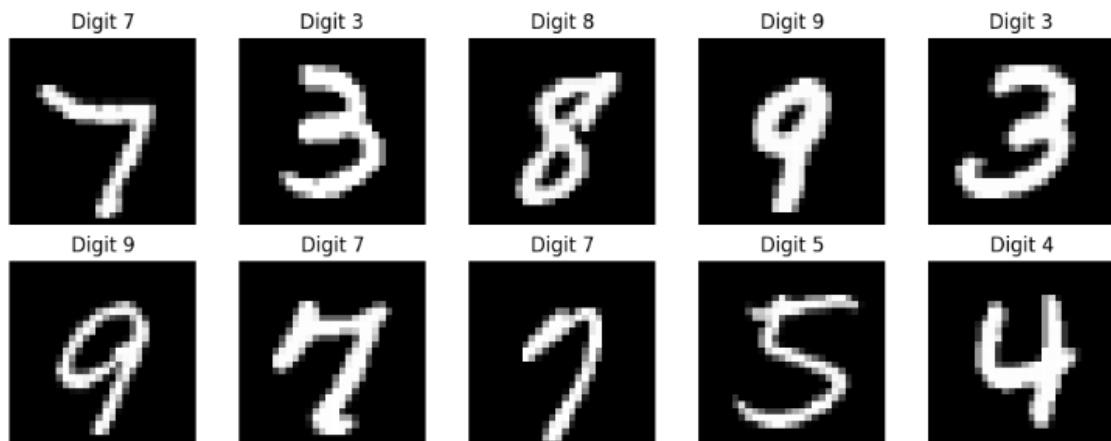
mnist_train[1].csv(text/csv) - 109640201 bytes, last modified: 8/11/2026 - 100% done

Saving mnist_train[1].csv to mnist_train[1].csv

Dataset loaded successfully!

Original Dataset Shape: (60000, 785)

Dataset used for training: (10000, 785)



Training Samples: 8000

Testing Samples: 2000

Training model...

Training completed!

...

Training Samples: 8000
Testing Samples: 2000

Training model...
Training completed!

=====

FINAL RESULTS

=====

Accuracy: 93.6 %

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.98	0.96	197
1	0.98	0.98	0.98	218
2	0.89	0.94	0.92	199
3	0.92	0.93	0.92	200
4	0.92	0.94	0.93	196
5	0.91	0.92	0.92	184
6	0.96	0.95	0.95	196
7	0.95	0.92	0.94	212
8	0.94	0.87	0.90	196
9	0.94	0.92	0.93	202
accuracy			0.94	2000
macro avg	0.94	0.94	0.94	2000
weighted avg	0.94	0.94	0.94	2000

Confusion Matrix:

```
[[194  0  0  0  0  2  0  0  1  0]
 [  0 213  1  0  0  0  1  1  2  0]
 [  2  1 187  0  1  0  1  5  1  1]
 [  0  0  4 186  0  5  1  1  2  1]
 [  2  2  0  0 185  0  3  0  0  4]
 [  1  1  0  5  3 170  0  0  2  2]
 [  5  0  2  0  0  3 186  0  0  0]
 [  0  0 10  1  2  0  0 196  1  2]
 [  1  1  3 10  5  4  1  0 170  1]
 [  1  0  2  1  5  3  1  3  1 185]]
```

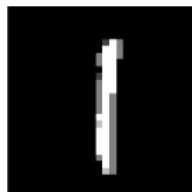
Actual: 8
Predicted: 8



Actual: 2
Predicted: 2



Actual: 1
Predicted: 1



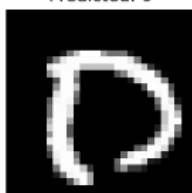
Actual: 2
Predicted: 2



Actual: 9
Predicted: 9



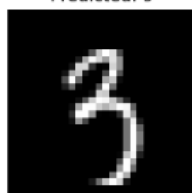
Actual: 0
Predicted: 0



Actual: 9
Predicted: 9



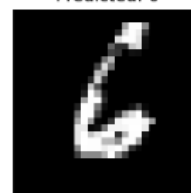
Actual: 3
Predicted: 9



Actual: 7
Predicted: 7



Actual: 6
Predicted: 6



Applications

1. Postal mail sorting.
2. Bank cheque processing.
3. Automatic form reading.
4. Digital document processing.
5. Postal ZIP code recognition.
6. Educational assessment systems.
7. Mobile handwriting recognition.
8. OCR (Optical Character Recognition).

Advantages

1. High digit recognition accuracy.
2. Reduces manual effort.
3. Fast and automatic prediction.
4. Improves document processing.
5. Handles large datasets efficiently.
6. Reduces human errors.
7. Easy to integrate into real-world applications.
8. Supports intelligent automation.

Conclusion:

A machine learning workflow provides a structured approach for classifying handwritten digits by progressing through stages such as data collection, preprocessing, feature extraction, model training, evaluation, and prediction. By learning patterns from labeled digit images, the model can accurately recognize new handwritten digits. This workflow improves accuracy, reduces manual effort, and is widely used in applications such as cheque processing, postal mail sorting, optical character recognition (OCR), and automated document digitization.